



Quanto

A multiplayer world built from live equity price feeds on
Robinhood Chain

Version 1.0 · August 2026

Robinhood Chain · chain 4663 · price data by Chainlink

This document describes a game and its design. It is not an offer to sell, or a solicitation to buy, any security or financial instrument, and it is not investment advice.

Abstract

Quanto is a persistent, browser-based multiplayer world in which every building corresponds to a real publicly-traded company, and that building's height is derived from the company's live share price. Prices are sourced from Chainlink oracles deployed on Robinhood Chain, an Ethereum Layer-2 network carrying tokenized US equities. Players lease floors within these towers, work shifts, collect during volatility events, and place signage. Ownership is expressed physically rather than through an interface: a leased floor renders as a lit window, so the aggregate glow of the skyline is a direct, continuously updated visualisation of the player economy.

The system's defining constraint is that market data drives world state rather than payout magnitude. Rewards are bucketed into four coarse volatility tiers and are strictly direction-neutral: no reward is larger for a security rising than for the same security falling by the same amount. This is a deliberate design decision with both gameplay and regulatory motivations, and is discussed at length in section 8.

This document describes the thesis, the world model, the economic design, the technical architecture, the fairness and threat model, the outstanding risks, and the project's current implementation status. It is a design document, not an offering.

1 · Thesis

Financial markets are almost universally rendered as charts. Charts are dense, efficient, and abstract; they are read at arm's length by people who have already learned to read them. They are also, for most people, inert. A line moving on a screen conveys magnitude but not consequence, and it conveys nothing at all about the simultaneity of a market. The fact that thousands of instruments are moving at once, in relation to one another, on a shared clock.

Games, by contrast, are spaces people inhabit. Spatial memory, peripheral awareness, and social presence are all things a world provides and a chart cannot. The premise of Quanto is that a market becomes legible in a categorically different way when it is a place: when volatility is weather you walk through, when a company's decline is a tower visibly shrinking beside you over the course of an afternoon, and when the trading day has a dawn, a peak and a dusk that you experience rather than observe.

This is not a new idea in the abstract. Software-visualisation research has represented codebases as cities for two decades, and the metaphor is durable precisely because buildings encode magnitude so naturally. What has not previously been possible is doing this with live, real, regulated market data inside a shared multiplayer world with a functioning economy, because the data layer required did not exist in a form that a game could consume cheaply and continuously.

1.1 Why now

Three properties had to converge. First, real equity price feeds needed to exist as a native on-chain primitive rather than as a bespoke oracle each project maintains itself. Second, reading those feeds needed to be free, because a world that costs money to render cannot be free to enter. Third, the underlying network needed transaction ordering that does not advantage the wealthiest participant, or any competitive mechanic built on it would degrade into an auction.

Robinhood Chain satisfies all three. It launched with Chainlink as its oracle provider and carries price feeds for roughly thirty tokenized US equities alongside major crypto pairs. Reading on-chain state, as

with any EVM network, requires no transaction and no gas. And it sequences transactions first-come-first-served, without the priority gas auctions that characterise most EVM chains.

2 · The world model

The city occupies a bounded plane divided into four districts, each grouping tickers the way market participants already group them: Tech Row for large-cap technology, Crypto Alley for 24/7 crypto pairs and crypto-adjacent equities, Moonshot Mile for high-beta speculative names, and Index Plaza for broad funds and metals. Approximately thirty-eight feeds are tracked at present. District membership is configuration, not code, and the roster is intended to grow.

2.1 Height derivation

A building's height derives from a logarithmic transform of its bonded ticker's price. A linear mapping is unusable across the ranges involved: an asset trading near \$63,000 would render approximately two and a half thousand times taller than one trading near \$24, producing a skyline consisting of one unreadable spike and thirty-seven flat slabs. The logarithmic base is then modulated by recent price movement, so that intraday change is visible as growth or contraction without destroying the proportional relationships that make the skyline readable at a glance.

Rendered height is eased toward its target rather than snapped. A price update therefore reads as a building growing over several seconds, which is both more legible and more pleasant than discrete jumps every polling interval.

2.2 Volatility as difficulty

Realised volatility is computed server-side from a rolling window of observed prices, as the standard deviation of log returns. It is the single most load-bearing derived quantity in the design: it sets yield tiers, influences floor pricing, determines shift payouts, and triggers storm events.

The consequence is that the market itself supplies the game's difficulty curve. A stable consumer-staples name is a low-risk, low-return holding that behaves predictably. A high-beta speculative name is contested, lucrative and erratic. No designer chose these characteristics; they are properties of the underlying companies, and they shift over time as real conditions shift.

2.3 Market hours

The tokenized-equity feeds publish on a 24/5 basis. Outside US market hours they hold their last published value indefinitely; the contract remains callable and returns a price, but that price stops changing. A naive integration treats this as a fault to be smoothed over or hidden.

Quanto instead treats it as the world's day-night cycle. When the feeds freeze, the skyline freezes with them, ambient fog closes in, lighting drops, and floor yield ceases. Crypto-bonded towers, whose feeds are genuinely continuous, remain live throughout. The result is a strategic asymmetry, continuous but volatile income in one district, higher but intermittent income elsewhere. That originates in market structure rather than in a tuning parameter.

Session state is determined by cross-referencing feed staleness against a trading calendar evaluated in the America/New_York timezone. Staleness alone is insufficient, because some feeds continue publishing during extended-hours trading; the calendar alone is insufficient, because a feed may be degraded during nominal market hours. Both signals are required.

3 · Core loop

Player activity is gated by an energy resource, CHARGE, which regenerates at one unit per five minutes to a ceiling of one hundred. Energy gating is a deliberate constraint against unbounded grinding: it makes the optimal strategy periodic engagement rather than continuous occupancy, which favours human players over automation and produces a rhythm compatible with a real trading day.

3.1 Shift work

A player may clock in at any tower and complete a short timing minigame. Payout scales with measured accuracy and with the tower's current volatility tier. Shifts require no ownership and no capital, providing an entry path for new players, and carry a per-building cooldown to prevent a single lucrative tower from dominating play.

3.2 Floor leasing

Floors are the core ownership primitive. A tower's floor count derives from its height, so more valuable companies offer more inventory. Purchasing a floor permanently lights one window on that tower for every player in the world.

Leased floors accrue yield continuously while the bonded feed is live, at a rate set by the volatility tier. A frozen feed yields nothing. This single rule is what gives the closing bell genuine economic weight rather than being a lighting change: at 16:00 New York time, most of the city's income stops.

3.3 Volatility storms

When a tracked ticker's realised volatility crosses a threshold, a storm event fires at that tower. Collectible shards spawn in the surrounding streets for approximately three minutes and the event is announced world-wide. Collection is first-touch, resolved authoritatively on the server, and free to participate in.

Storms are the only genuinely unscheduled content in the game. Nothing in the codebase decides when one occurs; the market does. This produces a live-service cadence, an unpredictable event that pulls concurrent players into one place, without a content treadmill or a live operations team.

3.4 Signage

Shards and currency combine to craft neon signs, which may be mounted on any tower where the player holds at least one floor. Signs are visible to all players and accrue a small yield from passing traffic. Their function is identity: they are the only mechanism by which a player writes something of their own onto the shared skyline, and the only reward in the system that is primarily social rather than economic.

4 · Technical architecture

The architecture's organising principle is a strict separation between responsiveness and settlement. The dominant failure mode of on-chain games is placing interactive state on-chain, which imposes confirmation latency and transaction cost on actions that must feel immediate. Movement is not a financial event and should not be priced as one.

4.1 Real-time layer

Movement, chat, ambient state, minigames and shard collection are handled by an authoritative game server running a fixed-rate simulation at twenty ticks per second. Clients transmit input commands, never positions. The server integrates those commands and replicates the resulting state.

To avoid the input latency that naive server authority produces, clients apply client-side prediction: input is simulated locally and immediately, buffered, and reconciled against authoritative state as it arrives, replaying any commands the server has not yet acknowledged. Remote players are rendered on a short interpolation delay so that their motion is always interpolated between two known states rather than extrapolated.

Client and server share a fixed simulation timestep of one sixtieth of a second. Deriving movement from frame delta instead would make speed a function of display refresh rate, allowing a high-refresh client to move proportionally faster. The server additionally caps commands consumed per tick, bounding how far a client can advance itself by flooding input.

4.2 Oracle layer

A server-side service reads all tracked feeds in a single multicall at a fixed interval of twenty seconds. It maintains rolling price history per ticker, derives realised volatility, computes normalised building heights, determines session state, and publishes the result into replicated world state.

Polling frequency is deliberately decoupled from player count: the cost of the oracle layer is constant whether five or five thousand players are connected. Reads are non-blocking with respect to the simulation loop, and a failed poll degrades to the last known good snapshot rather than stalling the world.

Critically, this entire layer is read-only. Consuming on-chain state requires no wallet, no transaction and no gas. This is why the world renders in full for visitors who have never connected a wallet, and why the marketing site can present live data without any backend of its own.

4.3 Contract layer

Four contracts constitute the on-chain system:

- OracleRouter resolves ticker symbols to Chainlink aggregators and mediates all price reads. It enforces per-feed staleness bounds, validates that answers are positive, and consults the L2 sequencer uptime feed with a grace period, since prices observed shortly after a sequencer outage may be arbitrarily stale. It exposes both a strict reverting read and a non-reverting variant, so that game logic can distinguish a legitimately frozen equity feed from an actual fault.
- BlockToken is the ERC-20 currency, with issuance capped per UTC day and minting restricted to authorised controllers.
- FloorDeed is the ERC-721 representing floor ownership. Token identifiers encode the ticker symbol and floor index directly, so location is derivable without a storage read and duplicate issuance of a given floor is structurally impossible. The identifier itself is the uniqueness constraint, enforced by the ERC-721 implementation.
- CityController is the sole state-changing entry point, handling floor purchases and batched wage settlement, with replay protection on settlement batches.

4.4 Settlement model

Earnings accrue continuously off-chain and settle on-chain in batches. Settling each individual wage accrual would be economically absurd and would reintroduce exactly the latency the architecture exists to avoid. Batches carry unique identifiers and are recorded as settled before any transfer occurs, making replay impossible.

5 • Economic design

5.1 Emission

Issuance is capped per UTC day, enforced in the token contract rather than in off-chain game logic. This bounds the consequences of any bug or compromise in the game server to a single day's issuance, and does so in a way that is independently verifiable rather than dependent on operational discipline. A compromised backend can misallocate a day's emission; it cannot inflate supply without limit.

5.2 Sinks

Floor purchases are burned rather than pooled into a treasury. Ownership is therefore a permanent sink, which prevents emission compounding into itself as holdings grow. Additional sinks include signage crafting, floor upgrades and cosmetic purchases. A system whose only sink is speculation is not an economy.

5.3 Yield tiering

Yield is bucketed into four volatility tiers with multipliers of 1x, 1.6x, 2.4x and 3.5x. The choice of coarse buckets over a continuous function is deliberate and is load-bearing in two independent respects.

Mechanically, coarse tiers produce legible decisions. A player can reason about "this tower is in the hot tier" without modelling a continuous payoff curve, and the tier is displayable as a single word. Continuous payoffs would reward optimisation over judgement and would make the game illegible to anyone unwilling to build a spreadsheet.

Structurally, a coarse, direction-neutral step function is materially different from a smooth payoff referencing security prices. This distinction is discussed in section 8.

5.4 Price discovery on floors

Floor prices scale with both the bonded company's price level and its current volatility. Expensive, volatile towers cost more because they yield more; cheap, stable towers are accessible entry points with correspondingly modest returns. Because both inputs are live, the cost of entering a position changes with market conditions, and the optimal allocation is not static.

6 • Fairness and threat model

6.1 Ordering

Robinhood Chain sequences transactions first-come-first-served without priority gas auctions.

Contested on-chain actions therefore resolve on arrival order rather than on willingness to outbid. This is a meaningfully different property from most EVM networks, where any contested opportunity degrades into a gas auction won by whoever is most capitalised or best connected to block producers.

6.2 Server authority

The server is authoritative over position, collision, collection and scoring. Clients declare intent, never outcome. Shard collection is resolved within a single tick with the shard removed from state before crediting, making double-claim impossible regardless of client behaviour or network timing.

6.3 Known limits

The timing minigame reports input timestamps to the server, which re-derives the score from parameters the client never receives authoritatively. A modified client cannot exceed the score achievable by perfect play, but perfect play is achievable by automation. This ceiling is inherent to any latency-tolerant timing mechanic: the client must know the target in order to render it, and network latency prevents purely server-side timing.

This is stated plainly rather than claimed as solved. Mitigation is treated as tuning, energy costs, cooldowns, and capping the marginal value of any single verb, rather than as a problem with a clean technical resolution. Sustained automation detection is a behavioural analysis problem and is out of scope for the current implementation.

6.4 Identity

Identity and holdings are separate systems with different lifetimes, and conflating them is a mistake with teeth. Identity is a verified account: the player authenticates with an email, a social provider or a wallet, and the server verifies the resulting token's signature before admitting the connection. Unauthenticated joins are refused outright, so there is no anonymous path into the world. That account is durable, survives devices, and decides which save loads. Holdings are a set of wallet addresses proved to the server by signature against a single-use, short-lived nonce. Signing proves key custody and authorises no transaction. A wallet belongs to exactly one account, enforced in the schema rather than by convention, and entitlements are resolved across every address an account has proved, because players routinely hold tokens in one wallet and play from another. Checking a single address refuses a genuine holder, which is the failure this design exists to prevent.

6.5 Names

Every account claims a username before entering the world, checked for availability as it is typed and unique on a case-insensitive index. Case insensitivity is a security property rather than a nicety: two accounts differing only in capitalisation are indistinguishable at a glance in chat, which is precisely the confusion a username exists to prevent. Names that impersonate operators, and the placeholder shape the server assigns before a player chooses, are reserved. Changes carry a cooldown, because a name identifies a player on standings that are frozen and published, and unrestricted renaming would let one player assume another's identity at the moment it mattered most.

7 • Scaling

The current architecture supports a single shared world instance. State is held in one process's memory and persisted to a local database; horizontal replication would produce divergent worlds rather than a larger one.

The binding constraint at scale is replication bandwidth rather than computation. Every connected client receives state deltas for every other player at the tick rate, which scales quadratically with concurrency. The correct remedy is interest management, replicating only entities within a player's relevant region, which is a substantial but well-understood piece of work and is the first item on the scaling path. Beyond that lie a networked presence layer, a shared database, and finally multiple instances behind session affinity.

Notably, the oracle layer does not participate in this scaling problem at all. Its cost is a fixed number of RPC calls per interval regardless of population.

8 · Risk

The most significant risk in this project is not technical, and it should be stated without hedging.

A token whose payouts vary with the price behaviour of real securities may be characterised by a financial regulator as a derivative instrument, irrespective of its presentation as a game and irrespective of the intentions of its authors. The relevant question is not whether something is described as entertainment; it is the economic substance of the payoff and the reasonable expectations of the people acquiring it.

8.1 Design mitigations

Several choices reduce this exposure deliberately rather than incidentally:

- Coarse tiering. Yield is a four-step function of realised volatility, not a continuous function of price return. A smooth payoff curve indexed to price is the structure that most closely resembles a swap.
- Direction neutrality. No reward is larger for upward movement than downward. There is no directional position to take, and therefore nothing resembling a bet on price direction.
- Volatility, not price. The economic input is how much a security moves, not where it moves to or what it is worth.
- No claim on the underlying. Holding floors confers no interest, economic or otherwise, in any referenced security, and no relationship with any issuer.
- In-game denomination. Rewards are denominated in an in-game unit with no redemption path.

These choices reduce exposure. They do not eliminate it, and nothing in this document constitutes legal advice or a legal conclusion.

8.2 Position

The project's stated position is that no cash-out path, exchange listing, or transferability for external value should be enabled prior to review by qualified counsel in every relevant jurisdiction, and prior to completion of an external security audit of the contracts described in section 4.3. Automated tests, however thorough, are not an audit, and the authors do not represent them as one.

8.3 Other risks

Oracle dependency is a single point of failure: the world's fidelity is bounded by feed availability and correctness. Degradation is handled by falling back to last known good state, but sustained oracle failure would render the world static. Retention risk is substantial and well-documented across the category; the mitigation strategy is to establish that the game retains players before any token exists,

rather than after. Concentration risk exists in the floor market, where early participants may accumulate disproportionate inventory in the most desirable towers.

9 · Prior art

Three lineages inform this design, and it is worth being explicit about what is borrowed and what is not.

9.1 Software cities

Representing codebases as cities is an established visualisation technique, dating to research in the early 2000s and popularised more recently by tools that render version control history as skylines. The insight these share is that building height is an exceptionally efficient encoding of magnitude: humans compare heights accurately, pre-attentively, and across large sets simultaneously, which is precisely what a numeric table fails at.

What these tools are not is inhabited. They are visualisations you orbit, not places you occupy, and they are static snapshots of historical data rather than live systems. Quanto takes the encoding and adds presence, persistence and a live data source.

9.2 Social worlds

The retention model draws on browser-based social worlds, persistent avatars, ownable and decoratable personal space, and public identity within a shared place. The durable lesson from that category is that people return for a space they have invested in and for the other people in it, not for mechanics in isolation. Ownership must be visible to others or it is merely a number in a menu, which is why floors are rendered as lit windows rather than displayed on an inventory screen.

9.3 On-chain games

The category's dominant failure mode is well documented: a token launches before the game is demonstrably enjoyable, speculative holders arrive, the token declines, and the apparent playerbase evaporates because it was never a playerbase. A secondary failure mode is architectural, placing interactive state on-chain, which makes ordinary play slow and expensive.

This project's response to the first is sequencing: establish retention before the economy carries external value. Its response to the second is the separation described in section 4. Neither response is novel. Both are simply applied rather than deferred.

10 · Onboarding and the first session

The single most consequential period in a browser game is the first sixty seconds, and the constraint is unusually tight here: a visitor arrives from a link, has no context, and is one click from leaving.

Three decisions follow. First, no wallet is required to play, and none is requested; the world is fully functional for an anonymous visitor because the oracle layer is read-only. Second, progress persists without an account, so a returning visitor is not punished for having declined to sign up. Third, every player begins with sufficient currency to take the central action, leasing a floor, within the first minutes, because the mechanic must be experienced rather than described to be understood.

The intended first-session arc is: arrive, recognise a company name, notice the tower is that company, lease a floor, watch a window light, and understand without being told that every other lit window

belongs to somebody. That comprehension moment is the product. Everything else is elaboration.

Wallet connection is positioned as an entitlement check rather than a sign-in, offered when a player wants the world to recognise what they hold. Connecting attaches an address and moves no progress whatsoever. An earlier design adopted whichever save that wallet had played on, which was coherent while the wallet was the identity and becomes a second identity system competing with the first once accounts exist: signing in as yourself and connecting a wallet that had played elsewhere would land another player's balance and floors on your account.

11 · Social design

A persistent world with no reason to notice other people is a single-player game with extra latency. Three mechanisms deliberately create interdependence.

Visible ownership. Because floors render as lit windows, the state of the economy is legible from the street. A newly bright tower is information: somebody decided that company was worth holding. This produces the imitation and competition that make markets social, without any interface for it.

Contested scarcity. Floor inventory per tower is finite and derived from price. Desirable towers fill, which creates genuine competition for position and gives early participation real meaning.

Synchronous events. Storms are the only mechanic that requires being present at a specific moment in a specific place. They exist to produce the experience of a crowd converging. The thing that makes a world feel populated rather than merely multiplayer. Their timing is set by market volatility, so they are unpredictable in a way scheduled content cannot be.

Signage layers identity on top of all three: the only mechanism by which a player writes something permanent and personal onto shared space.

12 · Roster evolution and governance

The tracked ticker roster is configuration rather than code, and is expected to change. Companies are added as feeds become available; a delisted or discontinued feed must be handled without orphaning the floors players hold in the corresponding tower.

The intended mechanism is that towers are retired rather than deleted. A tower whose feed is permanently discontinued stops accruing yield and freezes at its final height, with existing floor deeds remaining valid as artefacts. This preserves the property that a deed, once issued, is never invalidated by a decision the holder did not make.

Longer term, roster additions are a natural candidate for player governance, since the question "which companies should exist in this city" is one players have genuine opinions about and no privileged information is required to answer. This is noted as a direction rather than a commitment; governance introduces attack surface and is not worth implementing before there is a community to govern.

13 · Implementation status

The following reflects the state of the system as of this document's publication. Statements about future work are intentions rather than commitments.

World, districts, live oracle integration	Implemented; reading production feeds
Multiplayer, prediction and reconciliation	Implemented
Floors, shifts, storms, signage	Implemented and playable
Persistence and wallet sign-in	Implemented
Contracts	Written, unit-tested, not deployed
Interest management	Not begun; required beyond a few hundred concurrent players
Security audit	Not begun
Legal review	Not begun
Transferable \$BLOCK	Gated on the two rows above

14 · Appendix A, parameters

	Values as implemented. All are configuration and expected to move as the system evolves. These values are recorded here so that claims elsewhere in this document are checkable.
Oracle poll interval	20 seconds, single multical
Simulation tick	20 Hz network, 1/60 s fixed timestep
Remote interpolation delay	100 ms
Energy regeneration	1 unit / 5 minutes, cap 100
Shift cost	12 energy, 30-minute per-building cooldown
Volatility tiers	calm, normal, hot, extreme, 1x, 1.6x, 2.4x, 3.5x
Floors per tower	derived from height, bounded 6–40
Storm duration	approx. 3 minutes, 12–20 shards
Signage cost	8 shards + 120 \$BLOCK + 20 energy
Equity staleness bound	26 hours (accommodates a normal overnight close)
Crypto staleness bound	2 hours (a quiet 24/7 feed is a fault)
Sequencer grace period	3600 seconds after recovery

The two staleness bounds differ by an order of magnitude on purpose. A market published for eleven hours is behaving correctly. The market was shut. A period indicates a real failure. Treating both with a single threshold would state or fail to detect genuine outages.

15 · Appendix B, data sources

All prices originate from Chainlink aggregator contracts deployed on Robinhood Chain mainnet, chain identifier 4663. Feeds are consumed through the standard AggregatorV3Interface, and every tracked feed reports eight decimals.

Reads are performed through the canonical Multicall3 deployment, allowing the full roster to be retrieved in a single round trip. Feed proxy addresses are published by Chainlink and are not reproduced here, since they are subject to change and the authoritative list should always be consulted directly rather than copied.

The tokenized equity feeds price the token rather than the underlying share directly. Because dividends are reinvested through a multiplier, the token price tracks total return and diverges from the headline share price over time. This is correct for the purpose here. The game requires a consistent, live reference series, not a quotation, but it is noted because it means a tower's height will not exactly match a price quoted elsewhere, and that discrepancy is expected rather than a defect.

No proprietary or licensed market data is redistributed by this system. The game reads public on-chain state, as any participant may.

16 · Appendix C, rendering

The client renders an isometric world in two dimensions rather than a three-dimensional scene. This is a deliberate choice with three justifications.

The first is legibility. Height is the primary data channel in this world, and an isometric projection communicates relative height more reliably than a perspective camera, where distance and foreshortening confound the comparison. A player must be able to tell which of two towers is taller without moving, and a fixed projection guarantees that.

The second is reach. A two-dimensional renderer runs acceptably on hardware where a three-dimensional scene does not, and the target audience arrives from links on arbitrary devices rather than from a storefront on gaming hardware.

The third is that the entire visual identity is generated in code, facades, window grids, characters, signage and street surfaces are produced procedurally at runtime rather than loaded as assets. This removes an asset pipeline and an art dependency from the critical path, keeps the download negligible, and makes per-entity customisation free. The trade-off is a stylistic ceiling: procedural generation suits a neon aesthetic carried by light and colour, and would suit an illustrative one poorly. The generation layer is isolated behind factory functions so authored art can replace it without touching anything else.

Performance work concentrates on two techniques. Buildings share geometry and are drawn through instancing, so the skyline costs approximately the same regardless of how many towers exist. Lit windows are only drawn when a floor is actually owned, which means rendering cost scales with the size of the economy rather than the size of the map, an unowned city is nearly free to draw, and a heavily-owned one is exactly as expensive as it is interesting.

A level-of-detail threshold governs the transition to the zoomed-out view: below a certain scale, individual windows, characters and signage are dropped and towers reduce to glowing bars. This makes the full-city overview affordable and, incidentally, returns the display to something very close to

a conventional chart. The city and the chart being the same object viewed at different distances.

17 · Appendix D, glossary

\$BLOCK	The in-game currency. Presently has no cash value and no external transferability.
CHARGE	Energy. Regenerates over time and gates active verbs.
Floor	A unit of ownership in a tower, rendered as one lit window and represented on-chain as an ERC-721 deed.
Shard	A collectible spawned during volatility storms; the crafting input for signage.
Storm	A time-boxed event triggered when a tracked ticker's realised volatility crosses a threshold.
Tier	One of four volatility buckets that set yield multipliers.
Frozen feed	A price feed that has stopped publishing because its underlying market is closed. Normal for equities, a fault for crypto.
Session	Whether the US equity market is currently open, derived from feed staleness cross-checked against the trading calendar.
Reconciliation	The process by which a client corrects its predicted position against authoritative server state.
Interest management	Replicating only entities relevant to a given player; the primary scaling technique not yet implemented.
Settlement	Periodic on-chain recording of earnings accrued off-chain, performed in batches.

18 · Conclusion

The claim this project makes is narrow and, we think, defensible: that the substrate for a persistent world is more interesting than the invented economies games supply. It supplies volatility, rhythm, difficulty and unpredictability for free, and is legible to players who have never read a candlestick chart in their lives.

Everything else follows from taking that seriously. The closing bell market publishing. Storms are unpredictable because markets are. Some decisions some companies are. None of it required a designer to decide when to move.

The remaining work is not primarily technical. The world exists, the contracts are written. What remains is establishing that people want to play properly and with professional advice, the question of whether the economy

Those two questions are deliberately sequenced in that order. A world with an economic incentive is a foundation on which an economy can later be built. A world nobody returns to is not a foundation at all, and the category is sequencing costs time and forgoes the easiest available source of early order in which the outcome is worth having.

A final note on the data. Nothing in this system attempts to predict anything. It reads public prices, converts them into architecture and builds around inside the result. The market is used here as material, as a source of genuine surprise that no designer could author and no content scheduler could plan. The only order in which the outcome is worth having, is the whole of the idea.

This document describes a game and its design. It is not an offer to sell, or a solicitation to buy, or an offer to exchange, or a recommendation, or an investment instrument, and it is not investment advice. \$BLOCK is presently an in-game currency, not a security, and its transferability is limited. Statements about future development are intentions, not commitments.

